

提高级 CSP-S 第 19 套初赛模拟试题

一、单项选择题(共 15 题,每题 2 分,共计 30 分;每题有且仅有一个正确选项)

- 在 Linux 系统终端中,以下哪个命令不可以被用于创建一个新文件? ()。
A. touch B. echo C. create D. cat
- 以下哪个编译选项使 g++ 编译器开启所有警告信息? ()。
A. -Wall B. -g C. -O2 D. -std=c++14
- 以下关于 C++ 类的成员函数的表述中,哪一个是正确的? ()。
A. 构造函数可以有返回值,且可以被重载
B. 纯虚函数必须在派生类中实现,使得基类成为抽象类
C. 静态成员函数只能通过对象调用,而不能通过类名调用
D. 析构函数可以重载,以处理不同的清理需求
- 假设一个视频的时长为 10 分钟,比特率为 4Mbps,那么这个视频占用的存储空间约为()。
A. 4MB B. 40MB C. 280MB D. 2400MB
- 若元素 a, b, c, d, e, f, g 依次进栈,栈的大小为 3,以下哪一种出栈顺序是不可能的? ()。
A. c, b, a, d, e, f, g B. b, c, d, a, f, e, g
C. b, d, c, e, g, f, a D. a, d, e, b, c, g, f
- 对于一个拥有 5 个节点的有序二叉树,其共有多少种不同的形态? ()。
A. 12 B. 55 C. 273 D. 1428
- 对于一个二叉树,其前序遍历为 A, B, D, E, C, F, G,其后序遍历为 D, E, B, G, F, C, A,其可能对应的二叉树的数量是多少? ()。
A. 1 B. 2 C. 3 D. 4
- 对于以下 C++ 代码,其输出为()。

```
#include<iostream>
int main() {
    int a[1][2][3]{0, 1, 2, 3, 4, 5};
    std::cout<<* (* (* a + 1) + 1);
}
```


A. 2 B. 3
C. 4 D. 访问越界,没有输出
- 现共有六张扑克牌,点数分别为一到六,两个玩家会在开始游戏时各得到其中三张。游戏第一轮,玩家 A 会先出牌,然后玩家 B 必须出一张更大的牌来解牌;解牌后,第二轮玩家 B 先出一张牌,然后由玩家 A 出一张更大的牌来解牌;第三轮和第一轮规则一致。如果其中某一轮,有玩家无法解牌,那么他就会输掉;如果三轮游戏都能完成,那么就是平局。当两个玩家都采取最优策略时,有多少种发牌方式可以使得玩家 A 获胜? ()。
A. 10 B. 11 C. 12 D. 13
- 对于前缀表达式 $+ * 2 5 - 30 / 60 3$,下列哪个选项中的后缀表达式与其运算结果相等? ()。
A. $9 2 + 2 * 2 -$ B. $8 2 * 4 - 3 +$ C. $5 3 - 2 8 * +$ D. $6 2 / 3 * 2 *$
- 以下哪一个选项与 $(12.34)_8$ 的值相等? ()。
A. $(10.425)_{10}$ B. $(1010.0111)_2$ C. $(A.22)_{16}$ D. $(1001.1010)_2$
- 以下是无向图 G 的邻接矩阵,其有多少种不同的非空真子图是边双连通图? ()。
0 1 1 0 0 0
1 0 0 1 1 0
1 0 0 0 1 1

0 1 0 0 0 1

0 1 1 0 0 0

0 0 1 1 0 0

A. 1

B. 2

C. 3

D. 4

13. 对于以下 C++ 代码, 其输出为()。

```
#include<iostream>
#include<tuple>
union node {
    std::pair<int, double>x;
    std::tuple<int, int, long long>y;
};
int main() {
    std::cout<<sizeof(node);
}
```

A. 0

B. 12

C. 16

D. 28

14. 书架上从左到右共有 5 本书, 有多少种重新摆放的方法可以使得至多只有一本书在其原位? ()。

A. 88

B. 89

C. 90

D. 91

15. 以下哪个不是操作系统()。

A. NOI Linux

B. Mac OS

C. LaTeX

D. Android

二、阅读程序(程序输入不超过数组或字符串定义的范围, 判断题正确的填√, 错误的填×; 除特殊说明外, 判断题 2 分, 选择题 3 分, 共计 40 分)

1.

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 using ll=long long;
04 int main() {
05     int n;
06     cin>>n;
07     vector<ll>a(n);
08     for (auto &ai : a) {
09         cin>>ai;
10     }
11     auto a2=a;
12     sort(a2.begin(), a2.end());
13     auto last=unique(a2.begin(), a2.end());
14     vector<ll>cnt(last-a2.begin(), 0);
15     for (const auto &ai : a) {
16         int a2Idx=lower_bound(a2.begin(), last, ai)-a2.begin();
17         cnt[a2Idx]++;
18     }
19     sort(cnt.begin(), cnt.end());
20     ll ans=LLONG_MIN;
21     for (int i=1; i<=n; ++i) {
22         ll sum=0, num=i;
23         auto pos=lower_bound(cnt.begin(), cnt.end(), num);
24         while (pos != cnt.end()) {
```

```

25         sum+=num;
26         num<<=1;
27         pos=lower_bound(pos+1, cnt.end(), num);
28     }
29     ans=max(ans, sum);
30 }
31 cout<<ans<<endl;
32 return 0;
33 }

```

输入保证 $n \leq 2 \times 10^5, 1 \leq a_i \leq 10^5$

●判断题

- (1) 对于第 13 行代码, last 的变量类型为 long long *。()
- (2) 对于第 14 行代码, 将其更换为 `vector<ll>cnt(last-a2.begin())`; , 不会对程序运行结果产生任何影响。()
- (3) 程序的运行时间复杂度与 a_i 的取值范围无关。()
- (4) 当输入为 5 3 3 6 3 6 时, 其输出和输入为 5 9 1000000000 1000000000 9 9 时的输出一致。()

●单选题

- (5) 当输入为 10 1 2 1 1 3 3 3 4 5 4 时, 程序结束时, a2 可能的取值为()。(2 分)
 - A. 1 2 1 3 4 5 4 1 3 3
 - B. 1 2 3 4 5 4 3 1 3 1
 - C. 1 1 1 2 3 3 3 4 4 5
 - D. 5 4 3 2 1 1 1 3 3 4
- (6) 当输入为 18 2 1 2 10 2 10 2 2 1 10 10 10 10 1 1 10 10 时, 程序的输出为()。
 - A. 10
 - B. 18
 - C. 14
 - D. 9

2.

```

01 #include<bits/stdc++.h>
02 using namespace std;
03 const int MAXN=405;
04 int main() {
05     int n, m;
06     cin>>n>>m;
07     bool a[MAXN][MAXN];
08     int sRow[MAXN][MAXN], sCol[MAXN][MAXN];
09     for (int i=1; i<=n; ++i) {
10         string str;
11         cin>>str;
12         for (int j=1; j<=m; ++j) {
13             a[i][j]=str[j-1]=='1';
14             sRow[i][j]=a[i][j]+sRow[i][j-1];
15             sCol[i][j]=a[i][j]+sCol[i-1][j];
16         }
17     }
18     int ans=INT_MAX;
19     for (int i=1; i<=n; ++i) {
20         for (int j=i+4; j<=n; ++j) {
21             int cnt=0;

```

```

22         cnt+=(j-i-1)-(sCol[j-1][1]-sCol[i][1]);
23         cnt+=(!a[i][2]+!a[i][3])+(!a[j][2]+!a[j][3]);
24         cnt+=(sCol[j-1][2]-sCol[i][2])+
25             (sCol[j-1][3]-sCol[i][3]);
26         ans=min(ans,
27             cnt+(j-i-1)-(sCol[j-1][4]-sCol[i][4]));
28         for (int r=5; r<=m; ++r) {
29             cnt+=!a[i][r-1]+!a[j][r-1] +
30                 (sCol[j-1][r-1]-sCol[i][r-1]);
31             int newCnt=0;
32             newCnt+=
33                 (j-i-1)-(sCol[j-1][r-3]-sCol[i][r-3]);
34             newCnt+=(!a[i][r-1]+!a[i][r-2])+
35                 (!a[j][r-1]+!a[j][r-2]);
36             newCnt+=(sCol[j-1][r-1]-sCol[i][r-1])+
37                 (sCol[j-1][r-2]-sCol[i][r-2]);
38             cnt=min(cnt, newCnt);
39             ans=min(ans, cnt+(j-i-1)-
40                 (sCol[j-1][r]-sCol[i][r]));
41         }
42     }
43 }
44 cout<<ans<<endl;
45 return 0;
46 }

```

输入保证 $5 \leq n \leq 400, 4 \leq m \leq 400$, 第 11 行代码中的 str 为只包含 0 或 1 的字符串。

● 判断题

- (1) 对于第 34 行代码, 将其更换为 `newCnt+=4-a[i][r-1]-a[i][r-2]-a[j][r-1]-a[j][r-2]`; , 不会对程序运行结果产生任何影响。()
- (2) 设一次操作可以将一个 0 转换为 1 或一个 1 转换为 0, 这段程序希望计算给定的 01 矩阵中, 使其存在一个大小至少为 5 行 4 列的矩形区域, 其边界(包含四个角)均为 1, 其他区域均为 0 的需要的最少操作次数。()
- (3) 对于第 21 行至 28 行代码, 将其更换为 `int cnt=400*400; for (int r=4; r<=m; ++r) {`, 不会对程序运行结果产生任何影响。()

● 单选题

- (4) 对于以下的输入数据, 输出结果为()。

```

6 4
1000
0000
0110
0000
0001
1001

```

A. 11

B. 12

C. 13

D. 14

- (5) 对于以下的输入数据, 输出结果为()。

```

9 9
001010001

```

101110100
000010011
100000001
101010101
110001111
000001111
111100000
000110000

A. 5

B. 6

C. 7

D. 8

3.

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 string f1(string str) {
04     for (char ch='1'; ch<='9'; ++ch) {
05         string ans(str.length(), ch);
06         if (ans>=str)
07             return ans;
08     }
09     return str;
10 }
11 string f2(string str) {
12     string ans1=f1(str);
13     for (char ch1='0'; ch1<='9'; ++ch1) {
14         for (char ch2=ch1+1; ch2<='9'; ++ch2) {
15             bool flg=1;
16             for (int i=0; i<str.length(); ++i) {
17                 if (str[i]<ch2) {
18                     string ans2=str;
19                     if (ans2[i]<ch1) {
20                         ans2[i]=ch1;
21                     } else {
22                         ans2[i]=ch2;
23                     }
24                     for (int j=i+1; j<str.length(); ++j) {
25                         ans2[j]=ch1;
26                     }
27                     ans1=min(ans1, ans2);
28                 }
29                 if (str[i]!=ch1 && str[i] !=ch2) {
30                     flg=0;
31                     break;
32                 }
33             }
34             if (flg) {
35                 return str;
36             }
37         }
```

```

38     }
39     return ans1;
40 }
41 int main() {
42     string str;
43     int k;
44     cin>>str>>k;
45     if (k==1) {
46         cout<<f1(str)<<endl;
47     } else if (k==2) {
48         cout<<f2(str)<<endl;
49     }
50     return 0;
51 }

```

输入保证 $1 \leq n \leq 10^{18}$, $k=1$ 或 2 。

●判断题

- (1) 对于第 9 行代码,将其删去后可能会导致程序发生运行时错误。()
- (2) 对于第 12 行代码,将其更换为 `string ans1 = string(str.length(), '9');`,不会对程序运行结果产生任何影响。()
- (3) 对于第 14 行代码,将其更换为 `for (char ch2 = '0'; ch2 <= '9'; ++ch2) {`,不会对程序运行结果产生任何影响。()

●单选题

- (4) 当输入为 15 1 时,程序的输出为()。
A. 11 B. 15 C. 22 D. 23
- (5) 当输入为 234 2 时,程序的输出为()。
A. 222 B. 233 C. 234 D. 242
- (6) 当输入为 11772153280538483 2 时,程序的输出为()。
A. 11777111111111111111 B. 11777777777777777777
C. 11111111111111111111 D. 22222222222222222222

三、完善程序(单选题,每题 3 分,共计 30 分)

1. 稠密图最小生成树

给定 n 个点和 m 条边的无向连通稠密图 G ,求其最小生成树的权值和。输入满足 $n \leq 10^3$,点的编号从 0 开始。

```

01 #include<bits/stdc++.h>
02 using namespace std;
03 using ll=long long;
04 vector<pair<int, int>>s;
05 vector<pair<int, pair<int, int>>>e;
06 int find(int x) {
07     if (s[x].first==x)
08         return x;
09     else
10         return ①;
11 }
12 void merge(int x, int y) {
13     x=find(x);
14     y=find(y);

```

```

15     if (s[x].second>s[y].second) {
16         ② ;
17     }
18     s[y].first=x;
19     ③ ;
20 }
21 int main() {
22     int n, m;
23     cin>>n>>m;
24     for (int i=0; i<n; ++i) {
25         s.push_back({i, 1});
26     }
27     for (int i=0; i<m; ++i) {
28         int u, v, w;
29         cin>>u>>v>>w;
30         e.push_back({w, {u, v}});
31     }
32     int cnt=n;
33     ll ans=0;
34     while ( ④ ) {
35         vector<pair<int, pair<int, int>>> minE(n, {INT_MAX, {-1, -1}});
36         for (const auto &ei : e) {
37             int x=find(ei.second.first), y=find(ei.second.second),
38                 z=ei.first;
39             if (x!=y) {
40                 if (minE[x].first>z) {
41                     minE[x]=ei;
42                 }
43                 if (minE[y].first>z) {
44                     minE[y]=ei;
45                 }
46             }
47         }
48         for (const auto &ei : minE) {
49             if (ei.first!=INT_MAX && ⑤ ) {
50                 merge(ei.second.first, ei.second.second);
51                 ans+=ei.first;
52                 --cnt;
53             }
54         }
55     }
56     cout<<ans<<endl;
57     return 0;

```

(1) ①处应填()。

A. s[x].first

C. s[x].first=find(s[x].first)

B. s[x].first=find(s[x].first)

D. x=find(s[x].first)

(2) ②处应填()。

A. x=y

B. swap(x, y)

- C. swap(s[x].second, s[y].second) D. s[x].second=s[y].second
- (3) ③处应填()。
- A. s[x].second+=s[y].second
 B. s[x].second=max(s[x].second, s[y].second)
 C. s[y].second+=s[x].second
 D. s[y].second=max(s[x].second, s[y].second)
- (4) ④处应填()。
- A. cnt!=0 B. cnt>=0 C. cnt>0 D. cnt>1
- (5) ⑤处应填()。
- A. find(ei.second.first)!=find(ei.second.second)
 B. ei.second.first!=ei.second.second
 C. ei.second.first!=-1 && ei.second.second!=-1
 D. cnt>0

2. 区间更新单点查询

给定一个序列 an , 对其进行 q 次操作, 操作分为两种: $l\ r$ 表示将 $ai(l \leq i \leq r)$ 更改为其各数位数字之和, $2\ x$ 表示输出 ax 的值。输入满足 $q \leq n \leq 2 \times 10^5$

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 using ll=long long;
04 struct Node {
05     int l, r, num;
06     bool tag;
07     Node * lc, * rc;
08     Node(int l, int r)
09         : l(l), r(r), num(0), tag(false), lc(nullptr), rc(nullptr) {}
10 };
11 Node* build(int l, int r) {
12     Node* p= ①;
13     if (l==r) {
14         cin>> p->num;
15         p->tag=p->num<10;
16         return p;
17     }
18     p->lc=build(l, (l+r) / 2);
19     p->rc=build((l+r) / 2+1, r);
20     return p;
21 }
22 void updateNode(Node* p) {
23     int x=0;
24     while (p->num) {
25         x+=p->num % 10;
26         p->num /=10;
27     }
28     p->num=x;
29     p->tag= ②;
30 }
31 void updateRange(Node* p, int l, int r) {
32     if (p->l>r || p->r<l || p->tag) return;
```



```

33     if (p->l==p->r) {
34         updateNode(p);
35     } else {
36         updateRange(p->lc, l, r);
37         updateRange(p->rc, l, r);
38         p->tag= ③ ;
39     }
40 }
41 int query(Node* p, int idx) {
42     if ( ④ )
43         return 0;
44     else if (p->l==p->r)
45         return p->num;
46     else
47         return ⑤ ;
48 }
49 int main() {
50     int n, q;
51     cin>>n>>q;
52     Node* rt=build(1, n);
53     while (q-->0) {
54         int l, r, opt;
55         cin>>opt;
56         if (opt==1) {
57             cin>>l>>r;
58             updateRange(rt, l, r);
59         } else {
60             cin>>l;
61             cout<<query(rt, l)<<endl;
62         }
63     }
64     return 0;

```

(1) ①处应填()。

- A. nullptr B. new Node C. new Node() D. new Node(1, r)

(2) ②处应填()。

- A. p->lc->tag && p->rc->tag B. !(p->tag)
C. p->num!=0 D. x<10

(3) ③处应填()。

- A. p->lc->tag || p->rc->tag B. p->lc->tag && p->rc->tag
C. !(p->tag) D. p->lc->tag ^ p->rc->tag

(4) ④处应填()。

- A. p->l > idx || idx > p->r B. p->l >= idx || idx <= p->r
C. p == nullptr D. !p

(5) ⑤处应填()。

- A. idx < p->lc->r ? query(p->lc, idx) : query(p->rc, idx)
B. idx > p->rc->r ? query(p->rc, idx) : query(p->lc, idx)
C. max(query(p->lc, idx), query(p->rc, idx))
D. min(query(p->lc, idx), query(p->rc, idx))